

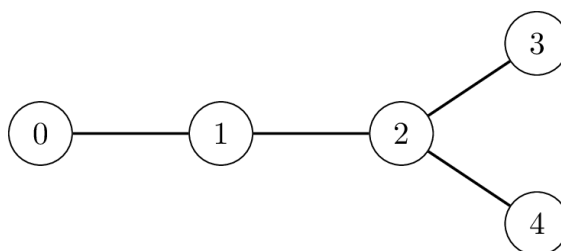
Migrácie

Kontrola Sauropodích Pozostatkov (KSP) je paleontologická organizácia, ktorá skúma, ako kedysi sauropody migrovali po Bolívii. Fosílie sauropodov boli nájdené na N rôznych miestach, ktoré paleontológovia očíslovali od 0 po $N - 1$ podľa ich veku: fosílie na mieste 0 sú najstaršie a tie na mieste $N - 1$ sú najmladšie.

Pre každé miesto okrem najstaršieho miesta 0 platí, že naň dinosaury prišli z práve jedného staršieho miesta. Formálnejšie, pre každé i od 1 po $N - 1$ vrátane existuje práve jedno miesto $P[i]$, odkiaľ dinosaury prišli na miesto i , a pre toto miesto platí $P[i] < i$. Opačným smerom to nemusí byť unikátne: z konkrétneho staršieho miesta mohli dinosaury prejsť na viacero novších.

Archeológovia modelujú každú migráciu ako **neorientovanú hranu** medzi vrcholmi i a $P[i]$. Všimnite si, že po takýchto hranách sa dá dostať z každého vrcholu do každého iného. **Vzdialenosť** medzi vrcholmi x a y je definovaná ako najmenší počet hrán, po ktorých treba prejsť počas cesty z x do y .

Na obrázku nižšie je príklad krajiny s $N = 5$ archeologickými miestami, pre ktoré platí $P[1] = 0$, $P[2] = 1$, $P[3] = 2$ a $P[4] = 2$. Vzďialenosť medzi miestami 3 a 4 je 2, keďže najkratšia cesta z vrcholu 3 do vrcholu 4 vedie po dvoch hranách.



KSP chce nájsť dvojicu vrcholov, ktorých vzdialenosť je maximálna. Všimnite si, že odpoveď na túto otázku nemusí byť jednoznačná. Napr. v príklade vyššie majú maximálnu vzdialenosť dve dvojice vrcholov: dvojice $(0, 3)$ a $(0, 4)$ obe majú vzdialenosť 3. Ak nastane takáto situácia, všetky dvojice s maximálnou vzdialenosťou budeme považovať za správne odpovede.

Problém je v tom, že KSP **nepozná** hodnoty $P[i]$. Rozhodli sa preto, že vyšlú do terénu Krtka, aby zistil všetko potrebné.

Krtko postupne navštíví miesta $1, 2, \dots, N - 1$ v tomto poradí. Pre každé i platí, že keď Krtko navštíví miesto i , postupne spraví obe nasledovné akcie:

- Pohrabe sa v zemi a zistí, z akého miesta $P[i]$ prišli dinosaury sem (na miesto i).
- Na základe všetkých doteraz nazbieraných informácií sa rozhodne, či z tohto miesta do centrály KSP nepošle žiadnu správu alebo pošle **práve jednu správu**.

Správy sa posielajú Starlinkom, ktorý je drahý, takže musia byť krátke. Presnejšie, každú správu musí tvoriť jediné celé číslo a to musí byť v rozsahu od 1 po 20 000 vrátane. Navyše dostal Krtko od účtovníčky nakázané, že za celý prieskum môže poslať **nanajvýš 50 správ**.

Tvojou úlohou je naprogramovať stratégiu, pomocou ktorej:

- Krtko vyberie, kedy a aké správy posilať
- Centrála KSP len na základe správ od Krtka (a informácie, z ktorého miesta bola ktorá správa poslaná) určí ľubovoľnú jednu dvojicu miest, ktorých vzdialenosť je maximálna.

Tvoj výsledný počet bodov bude navyše závisieť od toho, či Krtkovi pomôžeš ešte viac ušetriť. Presnejšie, tvoje body budú závisieť aj na tom, koľko správ Krtko pošle, aj na tom, aké najväčšie číslo použije.

Detaily implementácie

Vo svojom riešení musíš implementovať dve funkcie: prvú pre Krtka, druhú pre centrálu.

Krtkova funkcia musí vyzeráť nasledovne:

```
int send_message(int N, int i, int Pi)
```

- N : počet miest
- i : číslo miesta, ktoré Krtko práve navštívil
- Pi : hodnota $P[i]$, ktorú sa tam dozvedel

Túto funkciu počas jedného testu zavolá testovač presne $(N - 1)$ -krát, pričom pri týchto volaniach bude postupne platiť $i = 1, 2, \dots, N - 1$.

Táto funkcia by mala vrátiť celé číslo $S[i]$ popisujúce, čo má Krtko spraviť po zistení $P[i]$. Sú dve možnosti:

- Ak Krtko nemá poslať správu, $S[i]$ má byť 0.
- Ak Krtko má poslať správu, $S[i]$ má byť rovné jej hodnote. Musí teda platiť $1 \leq S[i] \leq 20\,000$.

Funkcia pre centrálu musí vyzeráť nasledovne:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : pole dĺžky N , v ktorom:
 - $S[0] = 0$.
 - Pre každé i od 1 po $N - 1$ vrátane je $S[i]$ celé číslo vrátené volaním Krtkovej funkcie `send_message(N, i, P[i])`.

Túto funkciu testovač zavolá pre každý test práve raz.

Návratovou hodnotou tejto funkcie by mala byť ľubovoľná dvojica miest (U, V) taká, že vzdialenosť medzi U a V je maximálna.

Pri skutočnom testovaní bude tvoj skompilovaný program obsahujúci obe vyššie uvedené funkcie **spustený presne dvakrát**.

- Pri prvom spustení programu:
 - Testovač postupne spraví $N - 1$ volaní tvojej funkcie `send_message`.
 - **Tvoj program si môže priebežne ukladať informácie získané počas týchto volaní a znova ich používať pri spracúvaní neskorších volaní.**
 - Testovač si uloží návratové hodnoty tvojej funkcie do poľa S .
 - V niektorých testoch je testovač **adaptívny**. To znamená, že hodnota $P[i]$, ktorú oznámi tvojej funkcii pri i -tom volaní `send_message`, môže závisieť na tom, aké akcie spravil Krtko počas predchádzajúcich volaní v rámci tohto testu.
- Pri druhom spustení programu:
 - Testovač práve raz zavolá tvoju funkciu `longest_path` a ako parameter jej odovzdá pole S obsahujúce hodnoty, ktoré pri prvom spustení vrátila Krtkova funkcia. Toto volanie `longest_path` nemá prístup k žiadnym iným informáciám, ktoré tvoje riešenie vypočítalo pri prvom spustení.

Obmedzenia

- $N = 10\,000$
- $0 \leq P[i] < i$ pre každé i od 1 po $N - 1$ vrátane

Podúlohy a hodnotenie

Podúloha	Body	Dodatočné obmedzenia
1	30	Medzi dvojicami s maximálnou vzdialenosťou je aspoň jedna, v ktorej je miesto 0 a nejaké iné miesto.
2	70	Žiadne ďalšie obmedzenia.

Nech Z je maximom poľa S a nech M je počet poslaných správ.

Uvažujme nasledovné tri podmienky:

- Niektorá hodnota v poli S je mimo povoleného rozsahu (napr. záporná).

- $Z > 20\,000$ alebo $M > 50$.
- Tvoja funkcia `longest_path` vrátila nesprávnu odpoveď.

Ak je splnená niektorá z týchto podmienok, dostaneš za príslušnú podúlohu 0 bodov a v CMS uvidíš verdikt `Output isn't correct`.

Ak všetky testy v podúlohe vyriešiš správne, dostaneš nejaké body.

V podúlohe 1 sa tvoje body určia takto:

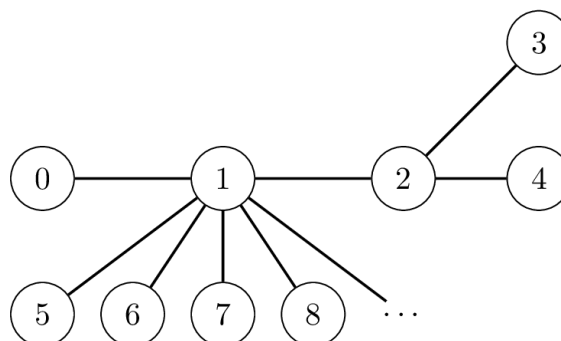
Podmienka	Body
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

V podúlohe 2 sa tvoje body určia takto:

Podmienka	Body
$5 \leq Z \leq 20\,000$ a $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ a $32 \leq M \leq 50$	40
$Z \leq 4$ a $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ a $M \leq 8$	70

Príklad

Nech $N = 10\,000$, pričom platí $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$ a pre všetky $i > 4$ platí $P[i] = 1$.



Povedzme, že Krtko používa nasledovnú stratégiu: vždy, keď sa po zavolaní `send_message` v jemu známej časti grafu zjaví nová dvojica (U, V) s maximálnou vzdialenosťou, pošle správu s hodnotou $10 \cdot V + U$.

Na začiatku platí, že dvojica s maximálnou vzdialenosťou je $(U, V) = (0, 0)$.

Počas prvého spustenia programu uvažujme nasledovné volania Krtkovej funkcie:

Volanie funkcie	(U, V)	Návratová hodnota (číslo $S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Vo všetkých nasledujúcich volaniach sa Krtko dozvie, že $P[i]$ je 1. Tým sa maximálna vzdialenosť už nikdy nezmení, a teda Krtko už nepošle žiadne ďalšie správy.

Pri druhom spustení programu spraví testovač práve jedno volanie funkcie pre centrálu, a to nasledovne:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Centrála sa pozrie na poslednú Krtkom poslanú správu. Tej hodnota je $S[3] = 30$. Z toho si v centrále odvodí, že Krtko posielal dvojicu $(0, 3)$, a tú vráti na výstup.

Na záver príkladu ešte podotkneme, že protokol, ktorý v ňom Krtko a centrála použili, nie je korektný, niektoré iné vstupy by nevyriešil správne.

Ukážkový testovač

Ukážkový testovač spraví počas jedného behu najskôr všetky volania `send_message` a potom aj jedno volanie `longest_path`. Toto správanie je iné ako u reálneho testovača.

Formát vstupu:

```
N
P[1] P[2] ... P[N-1]
```

Formát výstupu:

```
S[1] S[2] ... S[N-1]  
U V
```

Upozorňujeme ešte, že ukázkový testovač smiete používať s ľubovoľnou hodnotou N .